Efficient Data Placement Across Memory Tiers for High-Speed Network

CS614 Project Presentation

Arihant Kumar Dnyanesh Shinde Nikhilesh Joshi Rohit Somani
Yatharth Dangi

Endterm Report

Introduction & Motivation

- High-speed network I/O places significant pressure on the memory subsystem.
- Packet buffers allocated, accessed, and processed at line rate exacerbate this issue.
- **Core Question:** Can intelligent placement of NIC-received packet data across memory tiers improve throughput, latency, and CPU efficiency?
- **Goal:**
  1. Characterize performance costs of conventional packet reception and data copying paths.
  2. Design mechanisms to place received packet data into different memory tiers
  3. Evaluate performance gains regarding throughput and latency
  4. Analyze trade-offs between flexibility, complexity, and performance.

Background: Linux Packet Receive Path

- Packets transferred to memory via DMA, then processed through driver and kernel functions.
- Two distinct memory allocation pipelines in the e1000 driver:
  1. **RX Buffer Pre-allocation (Pipeline 1)**
  2. **Small-packet Copy Path (Pipeline 2)**

Pipeline 1: RX Buffer Pre-allocation

- Runs **before** packet arrival.
- Populates the receive ring using:
  - `e1000_alloc_rx_buffers`
  - Generic page allocators
- Ensures DMA-ready buffers are available when NIC receives a frame.

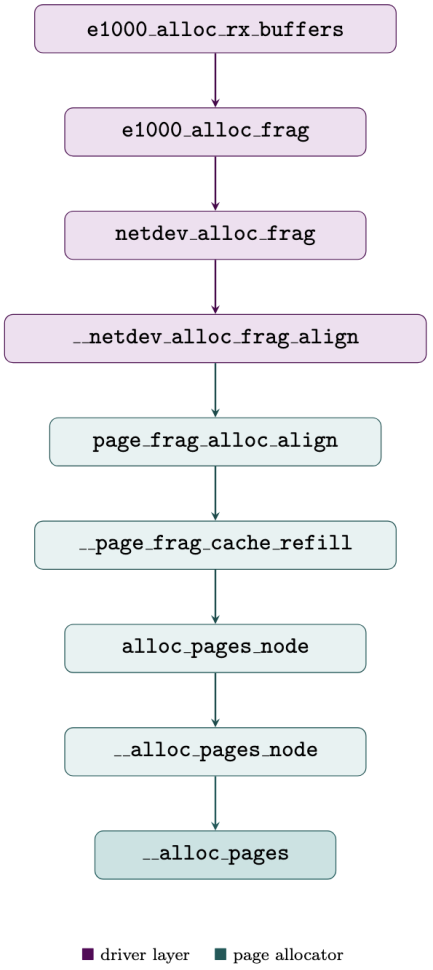


Figure: Buffer Pre-allocation pipeline

Pipeline 2: Small-Packet Copy Path

- Runs **after** packet reception.
- Applies to frames **below** the copybreak threshold.
- A new socket buffer (`sk_buff`) is allocated to:
  - Avoid pinning large pages in memory.
  - Allow the original page to be reused or freed promptly.

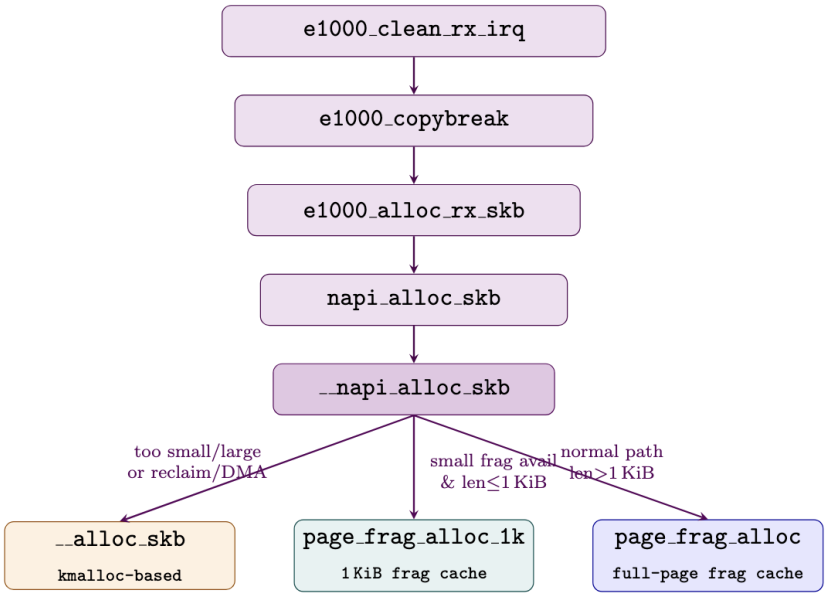


Figure: Copybreak pipeline

Implementation: Intercepting Allocations

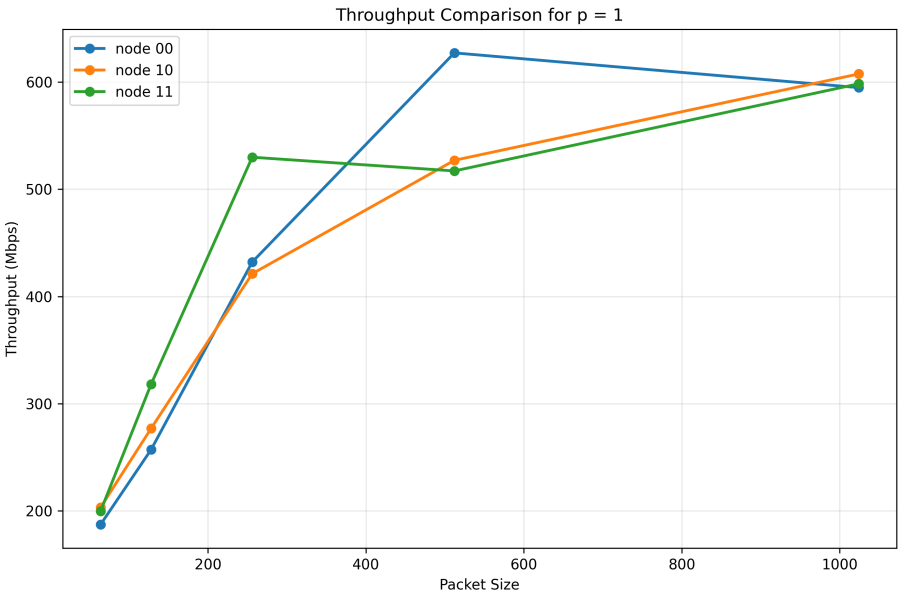
- **Kprobes Infrastructure:** Standalone modules (1 per policy) intercept memory allocation calls at runtime without modifying the kernel allocator.
- Overwrites function arguments in saved register state (e.g., target NUMA node).
- **Per-CPU Guard Flags:** Ensure only RX path allocations are targeted.
  - `in_rx_alloc`: Set during Pipeline 1.
  - `in_clean_alloc`: Set during Pipeline 2.
- No locking overhead and avoids false positives.

Static Policy

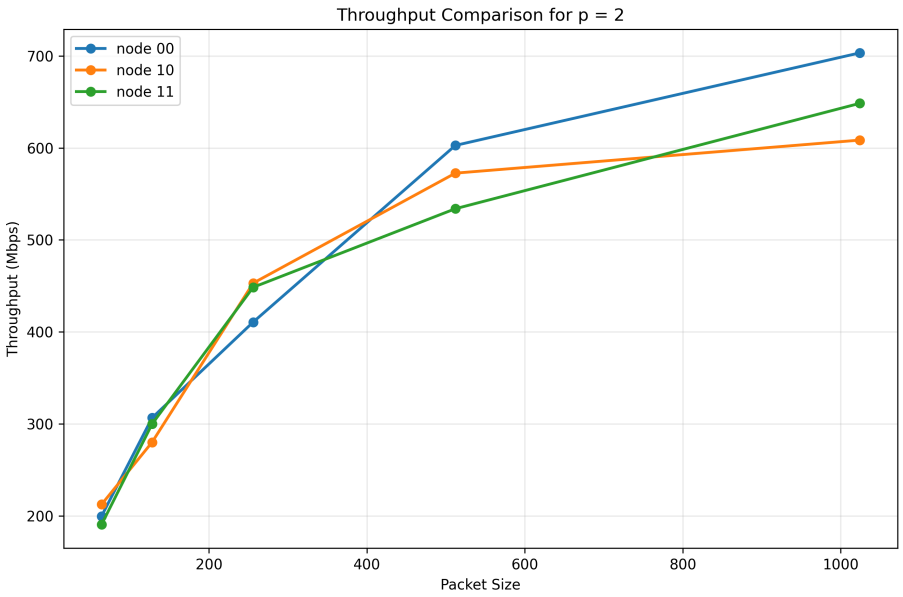
- Fixed placement configuration throughout the benchmark.
- **Sysfs Control Interface:** Variables exposed via `/sys/kernel/numa_force/`.
- Administrators can toggle forcing and target NUMA nodes for both pipelines independently:
  - Node 00: RX Buffer (0) and SKB→Data (0)
  - Node 10: RX Buffer (1) and SKB→Data (0)
  - Node 11: RX Buffer (1) and SKB→Data (1)

  Note: SKB→Data may be copied in copybreak or passed to upper stacks as it is.

Results: Throughput Comparison

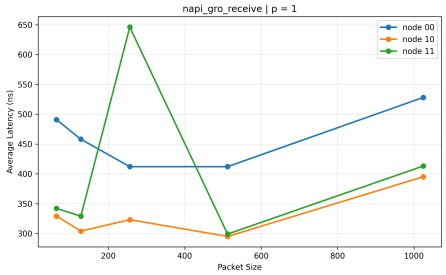


(a) Parallel Streams = 1

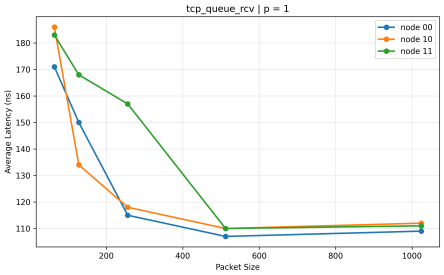


(b) Parallel Streams = 2

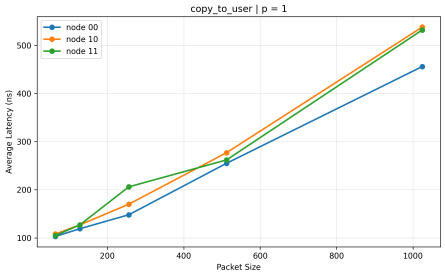
Results: Latency Comparison (Parallel Streams = 1)



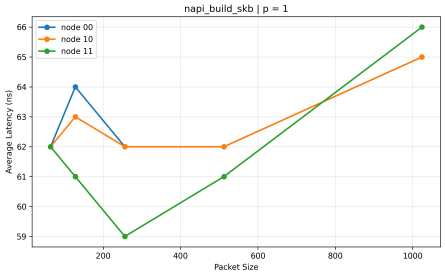
(a) napi_gro_receive



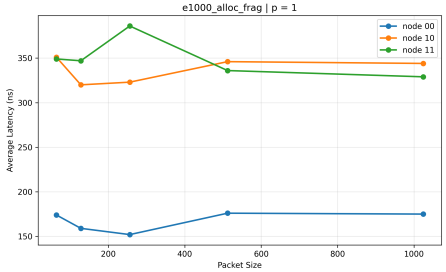
(b) tcp_queue_rcv



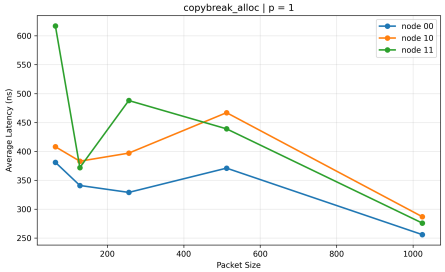
(c) copy_to_user



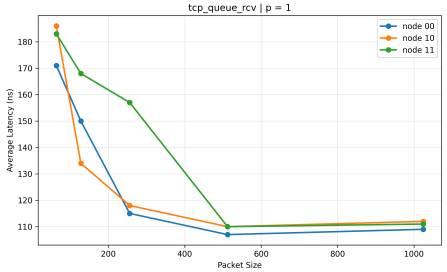
(c) napi_build_skb



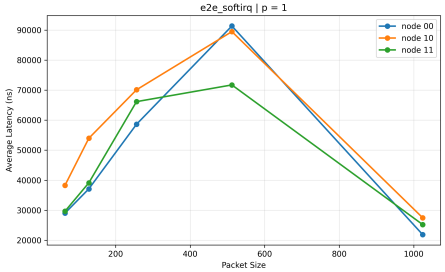
(d) e1000_alloc_frag



(e) copybreak_alloc

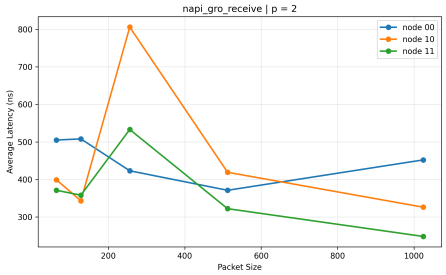


(f) tcp_queue_rcv

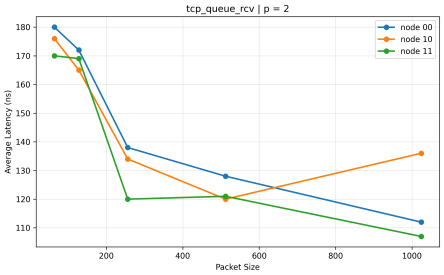


(f) e2e_softirq

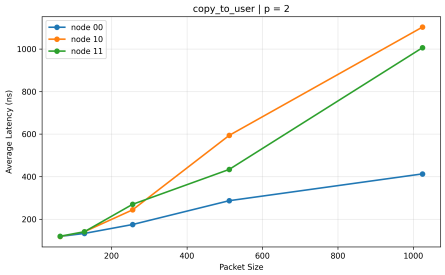
Results: Latency Comparison (Parallel Streams = 1)



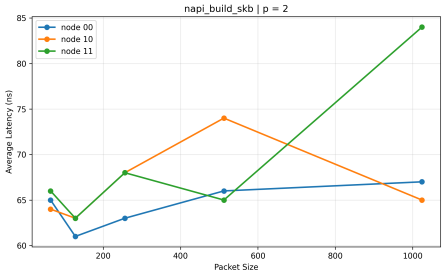
(a) napi_gro_receive



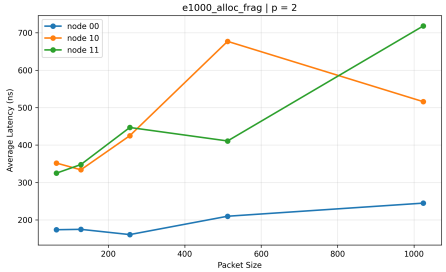
(b) tcp_queue_rcv



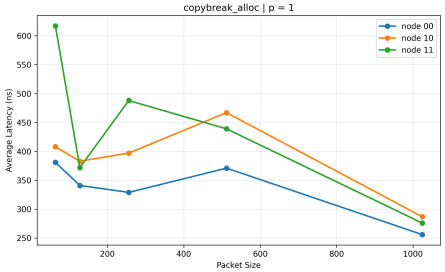
(c) copy_to_user



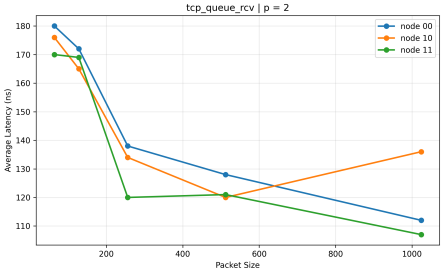
(c) napi_build_skb



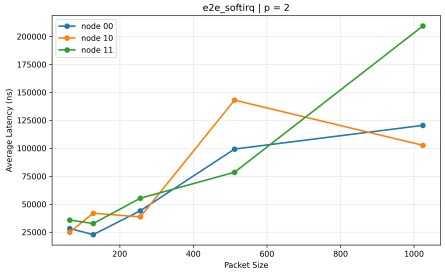
(d) e1000_alloc_frag



(e) copybreak_alloc



(f) tcp_queue_rcv



(f) e2e_softirq

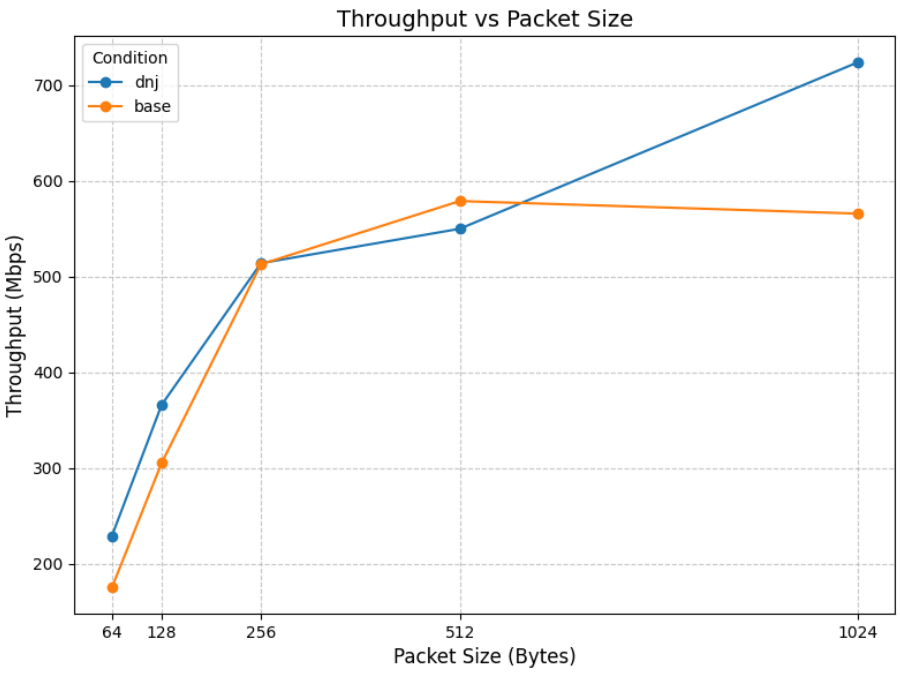
Motivation for Dynamic Policy 1

- **Size-Dependent Performance:** Throughput differences across memory tiers depend heavily on the size of the incoming packets.
- **Small Packets:** At small packet sizes, the throughput difference is relatively small. The performance penalty for allocating these packets to a remote node is minimal.
- **Large Packets:** As packet size increases, the throughput difference grows significantly. Allocating large packets to a remote node heavily penalizes overall throughput.
- **Policy Rationale:**
  - A fixed allocation strategy is sub-optimal under varying traffic.
  - We need a dynamic approach that tracks the real-time mix of small vs. large flows.
  - Small flows can be safely offloaded to the slow node (`SLOW_NID`), reserving the fast local node (`FAST_NID`) for large flows to maximize performance.

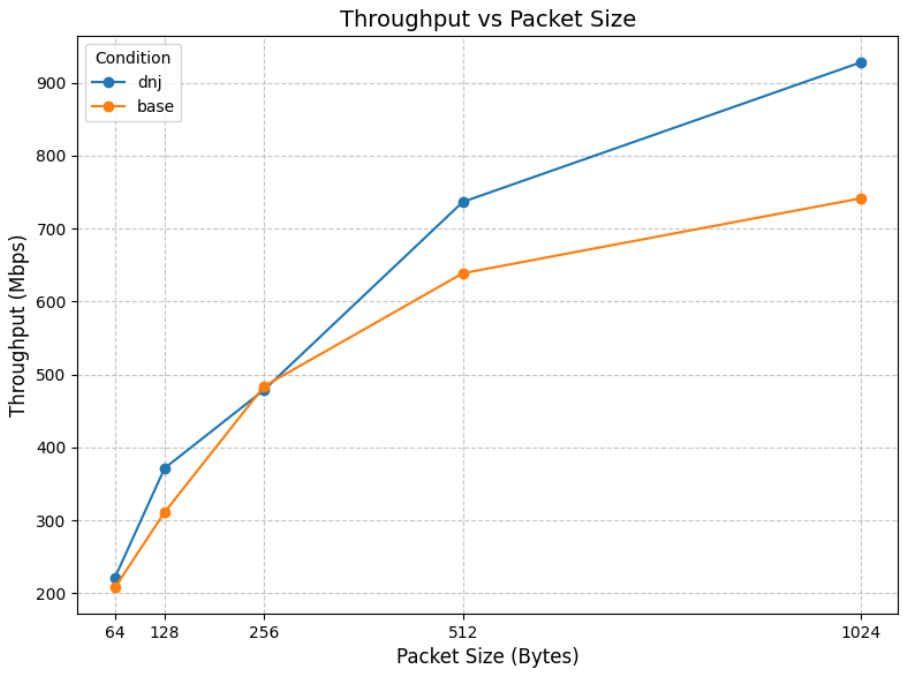
Dynamic Policy 1: Proportion-Based Allocation

- Adapts to the real-time mix of small vs. large packets.
- **Mechanism:** Uses per-CPU counters (`nr_small_packets`, `nr_big_packets`).
- **Decay and Reset:** Counters decay (by bit shifts) every 8192 packets to stay responsive to workload shifts. This is done so that there is some effect of history on allocation decision but not so much that it overpowers the new workload.
- **Decision Logic:**
  - If small flows dominate (`nr_small_packets > nr_big_packets`), allocations shift to the slow node (`SLOW_NID`).
  - If large flows dominate, allocations remain local (`FAST_NID`).
- Enforced using Kprobes on `__alloc_pages` and `__alloc_skb`.

Results: Throughput Comparison



(a) Parallel Streams = 1



(b) Parallel Streams = 2

Motivation for Dynamic Policy 2

- **Primary Goal:** Reduce NUMA misses during packet allocation.
- **Observation from prior policy:** Results showed a clear behavioural difference between large and small packets — this asymmetry motivated a packet size-aware, memory pressure-driven approach.
- **Problem with high memory pressure:**
  - When a node nears exhaustion, the kernel begins offloading large amounts of data to remote nodes.
  - This uncontrolled spillover causes a surge in NUMA misses.
- **Key Idea:** Proactively redirect small packets to a remote node *before* pressure becomes critical, thus avoiding the reactive, high-miss spillover entirely.

Dynamic Policy 2: Memory-Pressure-Aware

- Contextually evaluates local NUMA memory availability to help in allocation decisions.
- **Mechanism:**

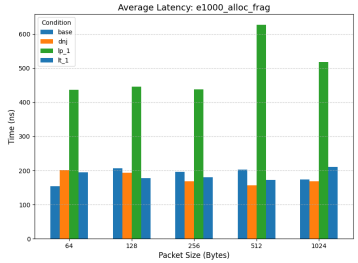
Free memory (%)	**Pressure level**	`numabreak` **(bytes)**
$\leq 10\%$	High pressure	2048
$\leq 18\%$	Mid-high pressure	1532
$\leq 26\%$	Mid-low pressure	600
$\leq 35\%$	Low pressure	300
$> 35\%$	Default (no pressure)	0

- If average packet size in NAPI window $<$ `numabreak`, allocations shifts to remote node.
- If packet size is less than numabreak (as well as copybreak), the processing of this packet takes place on remote node.

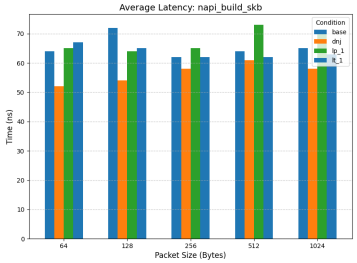
Experimental Setup

- **Network:** A TAP interface was configured between the host and the VM to expose the VM on a routable IP address.
- **Workloads:** TCP traffic generated via `iperf3`.
- **Variables:** Parallel streams ($p = 1, 2$) and varying packet sizes (64B to 1024B).
- **Metrics:** Throughput (Mbps) and Latency (ns).
- **Comparisons:** Baseline Static vs. Policy 1 (Proportion) vs. Policy 2 (Pressure-Aware).

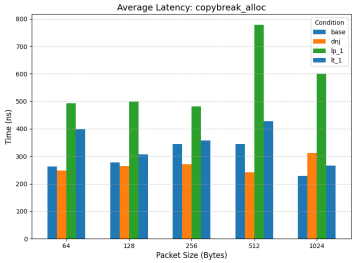
Results: Latency Comparison (Parallel Streams = 1)



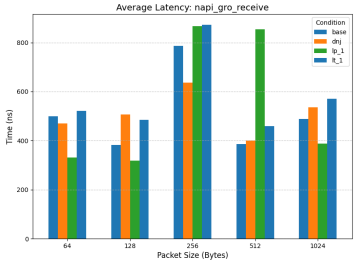
(a) e1000_alloc_frag



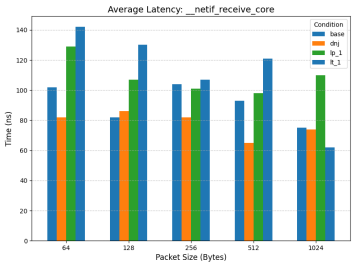
(b) napi_build_skb



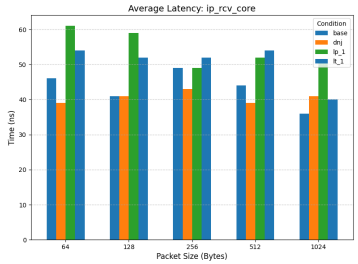
(c) copybreak_alloc



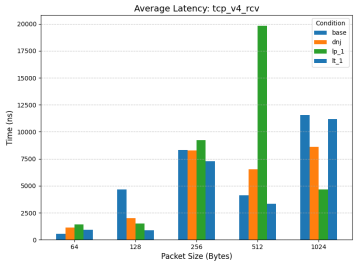
(d) napi_gro_receive



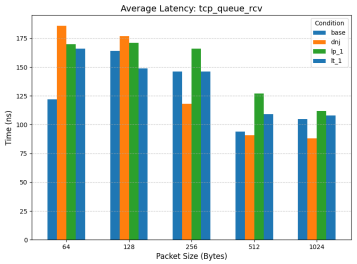
(e) $__netif_{r}eceive_{c}ore$



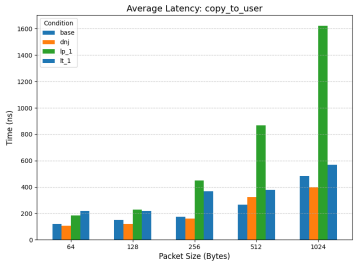
(f) ip_rcv_core



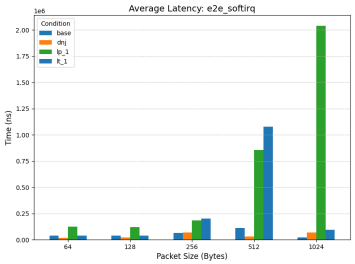
(f) tcp_v4_rcv



(c) tcp_queue_rcv

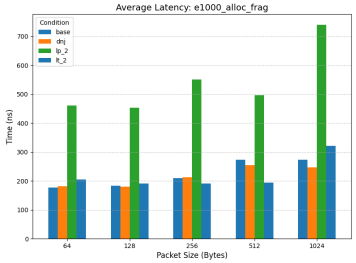


(d) copy_to_user

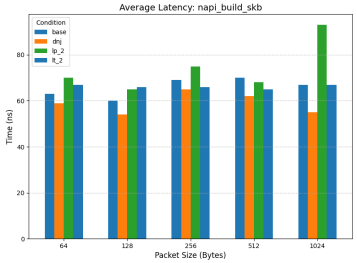


(f) e2e_softirq

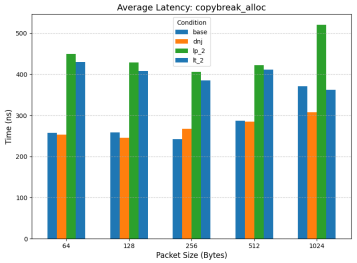
Results: Latency Comparison (Parallel Streams = 2)



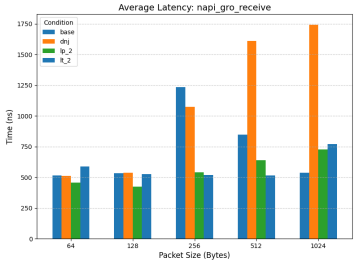
(a) e1000_alloc_frag



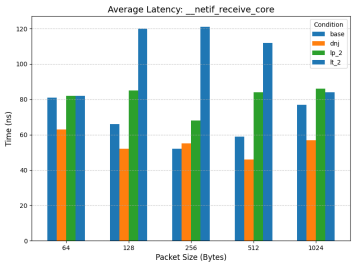
(b) napi_build_skb



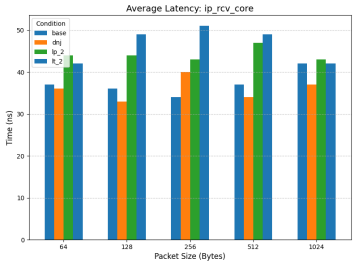
(c) copybreak_alloc



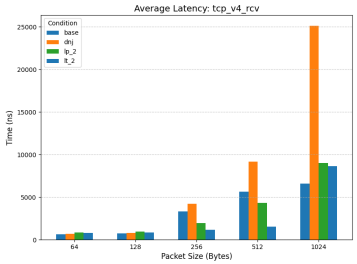
(d) napi_gro_receive



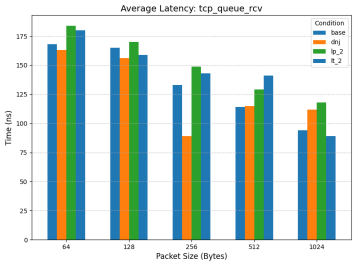
(e) $__netif_{r}eceive_{c}ore$



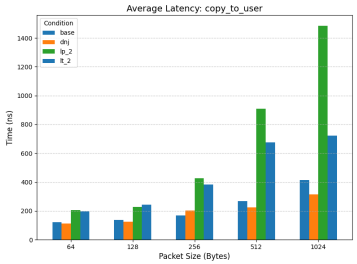
(f) ip_rcv_core



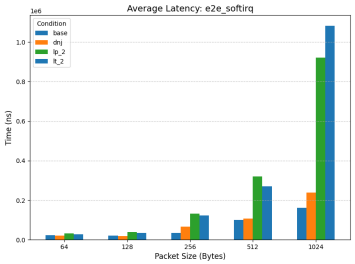
(f) tcp_v4_rcv



(c) tcp_queue_rcv



(d) copy_to_user



(f) e2e_softirq

Reasons for Inconsistencies

- **Emulated DMA Mechanism:** Instead of hardware bus mastering, QEMU translates guest DMA requests into software memory copies.
- **Throughput Dependency:** Network throughput is strictly bottlenecked by the rate at which the host CPU can execute these memory copies.
- **Host Pressure Sensitivity:** Because the "DMA controller" is actually a user-space thread on the host, any CPU contention, load pressure, or context switching on the host machine directly degrades the VM's network throughput.

Execution Stack:

```
qemu_send_packet
  ↓
e1000_receive_iov
  ↓
pci_dma_write
  ↓
address_space_rw
  ↓
flatview_write_...
  ↓
memmove(ram, ptr, len)
```

Reasons for Inconsistencies (contd.)

- **iperf3 Buffer Reuse:**
  - `iperf3` pre-allocates and reuses its send/receive buffers for the duration of the test.
  - As Node 0 fills up, new allocations should spill over to Node 1, triggering the pressure-aware policy — but since buffers are already reserved at startup, this spillover never happens.
  - The policy is never meaningfully exercised, leading to inconsistent throughput and latency measurements.
- **Host-side NUMA Interference:**
  - If NUMA misses occur on the host, the VM's allocation decisions become irrelevant.
  - The VM may direct data to a particular node, but the **final physical allocation is determined by the host**, independently of the guest policy.
  - There is no point in testing under such conditions, as the observed behaviour cannot be attributed to the policy under test.

Future Work

- **Unified Policy:** Combine traffic-type decisions with memory-pressure safety overrides.
- **Smoother Averages:** Implement running averages for packet sizes to prevent overreactions to micro-bursts.
- **Stress Testing:** Induce artificial memory shortages (e.g., `stress-ng`) to benchmark the protective limits of the pressure-aware policy.